

Terminal & Testing

```
# Run with input file
java Main < inputs/PE1.1.in

# Redirect output to file, then vim-diff
java PE1 < inputs/PE1.1.in > OUT
vim -d OUT outputs/PE1.1.out

# Inline diff (shows differences directly)
diff <(java Main < inputs/PE1.1.in) outputs/PE1.1.out
```

Common Signatures

```
public static void main(String[] args)
public boolean equals(Object o)
public int compareTo(Type o) // Comparable
public int hashCode()
```

Equality & Comparison

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    ClassName that = (ClassName) o;
    return n == that.n &&
        (s == null ? that.s == null : s.equals(that.s));
}

// Comparable<ClassName>
@Override
public int compareTo(ClassName o) {
    if (n != o.n) return n < o.n ? -1 : 1;
    if (s == null && o.s == null) return 0;
    if (s == null) return -1;
    if (o.s == null) return 1;
    return s.compareTo(o.s);
}
```

Event Handling

Event Pattern

Each event extends Event, stores context, and returns the **next** events from simulate().

```
class TicketSubmittedEvent extends Event {
    private RTSystem rtsys;
    private Ticket ticket;

    public TicketSubmittedEvent(double time,
        RTSystem rtsys, Ticket ticket) {
        super(time); // pass time to Event
        this.rtsys = rtsys;
        this.ticket = ticket;
    }

    @Override
```

```
public Event[] simulate() {
    try {
        this.rtsys.fileTicket(ticket);
    } catch (CongestionException e) {
        return new Event[] {
            new TicketDroppedEvent(this.getTime(), e, ticket)
        };
    }
    return new Event[] {}; // no follow-up events
}

@Override
public String toString() {
    return super.toString() + " " + this.ticket
        + " submitted " + this.rtsys;
}
}
```

Event Checklist

- Always **call super(time)** as first line of constructor.
- simulate() returns Event[] — next events to schedule; return new Event[]{} if none.
- Use this.getTime() (inherited) to pass time to follow-up events.
- Catch **checked exceptions** inside simulate() and return error events; let **unchecked** propagate.
- toString() should chain super.toString() for the timestamp prefix.
- Keep event classes small — delegate logic to domain objects (rtsys.fileTicket(...)).

Queue & Inheritance

Queue Signature

class Queue<T> implements Comparable<Queue<T>>

```
class Queue<T> implements Comparable<Queue<T>> {
    private Object[] items;
    private int first, last, len, maxSize;

    Queue(int maxSize) {
        this.maxSize = maxSize;
        this.items = new Object[maxSize];
        this.len = 0;
    }

    boolean isFull() { return len == maxSize; }
    boolean isEmpty() { return len == 0; }
    int length() { return len; }

    Queue<T> enq(T e) {
        if (isFull()) throw new RuntimeException("Full");
        if (isEmpty()) { first = 0; last = 0; }
        else { last = (last + 1) % maxSize; }
        items[last] = e;
        len++;
        return this;
    }

    @SuppressWarnings("unchecked")
    T deq() {
        if (isEmpty()) throw new RuntimeException("Empty");
        T val = (T) items[first];
```

```
        first = (first + 1) % maxSize;
        len--;
        return val;
    }

    @SuppressWarnings("unchecked")
    T peek() {
        if (isEmpty()) throw new RuntimeException("Empty");
        return (T) items[first];
    }

    @Override
    public int compareTo(Queue<T> other) {
        // < 0: this smaller; 0: equal; > 0: this larger
        return this.length() - other.length();
    }
}
```

enq Circular Logic

1. Check isFull().
2. If isEmpty(), set first = last = 0.
3. Otherwise, last = (last + 1) % maxSize.
4. Store element at items[last] and increment len.

Comparable Rules

- @Override public int compareTo(T other)
- Returns < 0 if this smaller; 0 equal; > 0 larger
- Compare queues by length(); strings via s.compareTo(t)

Generics & Wildcards (PECS)

Generic Class & Method

```
class Box<T> {
    private T v;
    public Box(T v) { this.v = v; }
    public T get() { return v; }
    public <U> Box<U> map(Transformer<T,U> f) {
        return new Box<>(f.transform(v));
    }
}

// Bounded: T must be Number or subclass
class NumBox<T extends Number> {
    private T value;
    public double getDouble() { return value.doubleValue(); }
}

// Multiple bounds: <T extends Shape & Drawable>
```

PECS

Producer Extends, Consumer Super

```
// ? extends T: READING (producer | getters)
void read(Seq<? extends Shape> src) {
    Shape s = src.get(0); // OK
    // src.add(new Circle()); // CANNOT add
}

// ? super T: WRITING (consumer | setters)
void write(Seq<? super Circle> dest) {
    dest.add(new Circle(5.0)); // OK
    // Circle c = dest.get(0); // CANNOT get specific type
}
```

- <?>: Unbounded (unknown type)
- <? extends T>: Upper bounded; **read-only**
- <? super T>: Lower bounded; **write-safe**

Generic Arrays

Cannot use new T[] (type erasure). Cast from Object[]:

```
@SuppressWarnings("unchecked")
T[] arr = (T[]) new Object[size];
```

Suppress only after justifying: array is private; only modified by typed setter; only returns subtypes of T.

Type Erasure

After compilation, generic types replaced by upper bound (Object). Cannot use new T(), new T[], T.class.

Exceptions & Factory Methods

Exception Hierarchy

- **Unchecked**: RuntimeException — programmer error; no throws needed
- **Checked**: Exception — must throws or try-catch

```
try {
    int result = riskyOperation();
} catch (SpecificException e) {
    // handle specific
} catch (Exception e) {
    // handle general
} finally {
    // always runs (cleanup)
}
// Checked: declare in signature
void readFile() throws IOException { ... }
```

Custom Exceptions

```
// Checked (extends Exception)
public class InvalidOperandException extends Exception {
    public InvalidOperandException() {
        super("Invalid operand provided for operation");
    }
    public InvalidOperandException(String msg) { super(msg); }
}

// Unchecked (extends RuntimeException)
public class CongestionException extends RuntimeException {
    public CongestionException(String msg) { super(msg); }
}
```

```
// Throwing and catching
void process(int x) throws InvalidOperandException {
    if (x < 0) throw new InvalidOperandException();
}
try {
    process(-1);
} catch (InvalidOperandException e) {
    System.out.println(e.getMessage());
}
```

Factory Methods

```
// Static factory (often named "of")
public static ClassName of(int n, String s) {
    return new ClassName(n, s);
}
```

```
}
```

Key advantage: Can return cached/existing instances (unlike constructors which always allocate new heap memory); can return subtypes; more descriptive name.

Useful Patterns

Null-safe & parse helpers

```
public String getSafe() { return (s == null) ? "" : s; }

int parseIntOr(String s, int d) {
    try { return Integer.parseInt(s); }
    catch (NumberFormatException e) { return d; }
}
```

Array Utilities

```
int sum(int[] a) {
    int s = 0;
    for (int i = 0; i < a.length; i++) s += a[i];
    return s;
}
double avg(int[] a) {
    return a.length == 0 ? 0.0 : (double) sum(a) / a.length;
}
int minIndex(int[] a) {
    int mi = 0;
    for (int i = 1; i < a.length; i++)
        if (a[i] < a[mi]) mi = i;
    return mi;
}
int[] copyOf(int[] a, int n) {
    int[] b = new int[n];
    int m = a.length < n ? a.length : n;
    for (int i = 0; i < m; i++) b[i] = a[i];
    return b;
}
// Insertion sort (stable, good for small arrays)
for (int i = 1; i < n; i++) {
    int v = a[i], j = i;
    while (j > 0 && a[j-1] > v) { a[j] = a[j-1]; j--; }
    a[j] = v;
}
```

Handy Utilities

- Math.abs(x), Math.max(a,b), Math.min(a,b), Math.pow(a,b), Math.sqrt(x)
- StringBuilder sb = new StringBuilder(); sb.append(...); sb.toString()
- System.arraycopy(src,0,dest,0,n) — fast array copy
- Character.isDigit(c), Character.isLetter(c), Character.toLowerCase(c)
- Integer.toString(i), Long.toString(l), Double.toString(d)

OOP Design Quick Tips

- **SRP**: Each class has one role. Split god objects.
- **Encapsulate**: private fields; expose minimal API; return copies for mutable arrays.
- **Composition > Inheritance**: unless clearly is-a.
- **Immutability**: final fields, no setters where possible.
- **equals/hashCode**: override both if used in sets/maps.

- **Tell, Don't Ask**: circle.isLargerThan(5) not circle.getRadius() > 5.
- **Avoid static overuse**: global state hurts testability.
- **Silent swallow**: never catch(Exception e) {}; log or rethrow.

Access Modifiers

- **public**: everywhere; **protected**: package + subclasses; **(default)**: package only; **private**: within class
- **static**: class-level (shared); **final**: cannot reassign; **final class**: cannot extend; **final method**: cannot override
- **Override**: same signature, covariant return OK, cannot reduce visibility

Java Style Rules

Identifiers

- UpperCamelCase for classes; lowerCamelCase for methods
- ALL_CAPS_SNAKE_CASE for constants (static final)
- One variable per declaration; arrays: A[] arr

Statements

- Switch needs default case; always use @Override
- Don't ignore caught exceptions; no wildcard imports
- Import statements in alphabetical order

Debug Checklist

- Compile error? Fix first error, then re-run.
- Off-by-one? Check loop bounds and indices.
- NullPointerException: which var is null? Add null guards.
- ArrayIndexOutOfBoundsException: check arr.length and indices.
- Wrong output: trace with System.out.println intermediate values.

Vim Essentials

- Save/quit: :w, :q, :wq, force :q!
- Insert: i before, a after, o/O new line; Undo u, Redo Ctrl-r
- Move: h j k l; word w e b; line 0 \$; file gg/G
- 5j down 5 lines; 10dd delete 10 lines
- Copy/Paste: yy yank line; p paste after; P paste before
- Delete: dd cut line; D delete to end of line
- vsp [file]: open file in vertical split
- Search: /pattern fwd, ?pattern back, n/N next/prev
- Replace line: :s/old/new/g; whole file: :%s/old/new/g; confirm: add c
- Case-insensitive: :%s/\cold/new/g or :set ignorecase smartcase
- Alt delimiter: :%s#path/old#path/new#g

```
" ~/.vimrc
set number
set relativenumber
set mouse=a
```

String.format Guide

- %s string, %d int, %f float, %b bool, %c char, %x hex
- Width: %8d; precision: %.2f; both: %8.2f
- Flags: - left align, 0 zero-pad, + show sign, , grouping

```
String.format("%s scored %d/%d (%.1f%%)",
    name, got, total, got*100.0/total)
String.format("Hex: 0x%08X", n)
String.format("%-10s | %8.2f", item, price)
```

Exam Strategy

- Build minimal working solution first (happy path), then add error handling.
- Keep code simple and well-named; markers reward clarity.
- Comment only when necessary; short inline comments ok.